

chapter_4_notes.pdf
PDF

我在复习计算机网络的课程。附件是第4章的slides,, 请你按照课件的思路, 详细的给我过。不要一下子输出所有, 而是输出一节解释, 让我理解一下, 并给我提问的机会。如果我没问题了再继续

已思考 4 秒 >

好, 我们先从 **4.1 Network layer overview (数据平面 vs 控制平面)** 这一节开始, 按课件的叙事顺序走一遍。  chapter_4_notes

4.1 网络层在干什么

网络层的核心工作可以一句话概括:

把发送端主机的 **transport segment (TCP/UDP 段)** 封装成 **IP datagram (IP 数据报)**, 在一跳一跳的路由器之间转发, 最后在接收端主机把 **datagram** 交给传输层。

 chapter_4_notes

注意课件强调的一点: **网络层协议存在于“每一台主机 + 每一台路由器”里**。路由器会检查每个经过它的 **IP 数据报头部字段** 来决定怎么转发。  chapter_4_notes

两个关键功能: forwarding vs routing

课件把网络层拆成两个动作, 并用“开车”打比方:  chapter_4_notes

1) Forwarding (转发)

- 发生地点: 每一台路由器本地
- 做的事: 把一个到达输入端口的包, 送到正确的输出端口
- 类比: 在一个高速路立交桥里, 怎么选对出口 (很局部、很快)

2) Routing (路由选择)

- 发生范围: 全网级别
- 做的事: 决定从源到目的地走哪条路径 (经过哪些路由器)

- 类比：出发前规划整趟行程路线

你可以把它记成一句对仗：

- **routing** 决定“走哪条路”（路径）
- **forwarding** 决定“这一跳怎么走”（端口/下一跳）

数据平面 vs 控制平面（这章的主线）

课件接着把上面两件事映射到两个“平面”： chapter_4_notes

Data plane（数据平面）

- 本地、每台路由器各自执行
- 输入：一个到达的 datagram（看它的头部字段）
- 输出：选定一个输出端口并转发
- 你可以把它想成：“查表 + 执行动作”（后面会叫 match-plus-action）

Control plane（控制平面）

- 全网逻辑
- 负责：算出/维护转发表（forwarding table），并让每台路由器的数据平面能“查到正确的表”
- 课件列了两种控制平面实现方式： chapter_4_notes
 1. 传统方式：每台路由器里跑路由算法/协议（RIP/OSPF/BGP），大家交换信息，分布式地产生表
 2. SDN 方式：逻辑上集中（remote controller 计算并下发表），路由器更像“只负责转发的执行者”

到这里你只要抓住：**第 4 章重点讲 data plane（怎么转发）；第 5 章讲 control plane（表怎么来的）。**  chapter_4_notes

你现在可以自检的 3 个小问题（用来确认你真的“抓住区别”）

1. “查 forwarding table 选输出端口”属于 data plane 还是 control plane？
2. OSPF/BGP 这类协议主要在解决 forwarding 还是 routing？
3. SDN 的 remote controller 主要在做哪件事：转发包，还是计算/分发表？

你先回答/提问都行。你确认这一节没问题后，我再按课件顺序继续下一节（通常是 **Network service model** 或直接进 **4.2 router 内部结构**）。



1. data plane; 2. routing; 3. 计算分发

已思考 11s >

答得很准  那我们按课件顺序继续下一小节：**Network service model**（网络层能“承诺”什么服务）。  chapter_4_notes

4.1（续）Network service model：网络层这条“通道”到底给你什么保证？

课件先抛一个问题：网络层把 datagram 从 sender 运到 receiver，这条“channel”可以提供什么服务模型？  chapter_4_notes

1) 先想象：理论上可以承诺的服务有哪些？

课件给了两类例子：对“单个数据报”的、对“一条流”的。  chapter_4_notes

对单个 datagram 可能的服务：

- guaranteed delivery（保证送达）
- guaranteed delivery with < 40ms delay（保证送达且时延上界，比如 <40ms）

 chapter_4_notes

对一个 flow（连续一串 datagrams）可能的服务：

- in-order delivery（保证按序）
- guaranteed minimum bandwidth（保证最小带宽）
- restrictions on changes in inter-packet spacing（限制抖动/包间隔变化）

 chapter_4_notes

你可以把它理解成：网络层如果“想当一个靠谱的快递公司”，可以给你承诺：**不丢、准时、按序、保底带宽、低抖动**。

2) 现实：不同网络架构选了不同的“服务模型”

课件用一个表直接对比 Internet vs ATM（两种 ATM 服务：CBR/ABR）。重点结论是：

Internet 的网络层服务模型 = best effort（尽力而为）。  chapter_4_notes

表里逐项告诉你 best effort 的含义（不是“努力帮你做”，而是“几乎不做承诺”）：

 chapter_4_notes

- Bandwidth: none（不承诺带宽）
- Loss: no guarantee（可能丢包）
- Order: no（不保证按序）
- Timing: no（不保证时延上界）
- Congestion feedback: no（不显式反馈拥塞；只能靠“丢包/延迟变大”去推断）

 chapter_4_notes

对比 ATM：

- CBR 会提供更强的时延/有序等保证
- ABR 会提供 guaranteed minimum bandwidth，并且有 congestion feedback 机制

 chapter_4_notes

你必须抓住的“课程主旨”

这一节不是要你背 ATM，而是要你形成一个非常关键的直觉：

Internet 网络层选择 best-effort，所以“可靠性、按序、拥塞控制的精细行为”很多责任被推到了端系统（尤其是 TCP）去完成。

自检 2 个小问题（你答完我们就进 4.2 路由器内部）

1. “best effort” 具体意味着网络层对 **loss / order / timing / bandwidth** 分别做不做保证？（用一句话概括也行）  chapter_4_notes
2. 表里写 Internet 的 congestion feedback 是 “no (inferred via loss)”，你怎么用自己的话解释 “inferred via loss”？  chapter_4_notes

来源



1. 都不做保证
2. 通过丢包的情况有相关的算法来判断 congestion 情况；问题：这个 atm 是什么

已思考 9s >

你前两题回答都对，而且你对“inferred via loss”的解释也对（丢包/超时/RTT暴涨 → TCP 端推断拥塞）。  chapter_4_notes

你问的 ATM 是什么：

ATM（这里不是取款机）

课件里这个 **ATM = Asynchronous Transfer Mode（异步传输模式）**，是一种 90 年代很重要的电信/广域网技术，曾经和 IP/Ethernet 竞争过，主打 **面向连接 + QoS（服务质量保证）**。  维基百科 +1

它跟今天 Internet 的“best effort”对比，是为了强调：

Internet 网络层基本不承诺；而 ATM 这类架构会在网络层/交换层就提供更强的承诺（带宽、时延、拥塞反馈等）。  cis.temple.edu +1

ATM 的几个关键点（你只要记这几个就够用）

- **固定长度 cell：53 bytes**（而不是 IP 那种可变长 packet），目的是降低排队抖动、对语音视频更友好。  维基百科
- **connection-oriented（面向连接）**：数据发送前要先建立“虚电路”（virtual circuit）。  维基百科 +1
- **QoS / traffic classes**：比如课件表里的 CBR/ABR/UBR 等服务类别（CBR≈恒定比特率；ABR≈可用比特率且通常有最小保证；UBR≈不保证）。  维基百科 +1
- 实际上后来 **Ethernet 提速 + IP 生态胜出**，ATM 在很多场景逐步被替代，但概念很适合拿来教学对比。  维基百科 +1

如果你对 ATM 没别的疑问，我们就进入下一节（通常是 **4.2 Virtual-circuit networks vs Datagram networks**，也就是：ATM 代表 VC 网络，Internet 代表 Datagram 网络），这会直接解释“为什么 Internet 能 best-effort 但还能跑得很好”。

来源

     ...  来源

继续，应该到了what's inside a router部分了

已思考 16s >

好，我们进入 **4.2 What's inside a router**，先讲这一节的第一块：**Router architecture overview + 输入端口在干什么**（先把“路由器内部流水线”搭起来）。  chapter_4_notes

4.2 路由器内部总体结构（先认清 4 个模块）

课件这张图的核心是：一个路由器可以被拆成四大块：  chapter_4_notes

1. **Input ports**（输入端口）
2. **Switching fabric**（交换结构/交换矩阵）：把包从某个输入端口“搬运”到某个输出端口
3. **Output ports**（输出端口）
4. **Routing processor**（路由处理器）

同时，它把“数据平面 vs 控制平面”落到硬件实现上：  chapter_4_notes

- **数据平面 (forwarding)**：主要在硬件里跑（input/output ports + switching fabric），要求“快”。
- **控制平面 (routing/management)**：主要在软件里跑（routing processor），算路由、维护状态，要求“对”。
- **forwarding table** 是 routing processor 算出来的，然后 push 到各个 input port，让输入端口能本地快速查表转发。  chapter_4_notes

直觉：包来了不能等 CPU 算路由；CPU 先把“转发表”算好发下去，硬件只负责照表执行。

输入端口的工作流水线（包一进来发生什么）

课件把 input port 画成一个从下到上的小流水线：  chapter_4_notes

Step A: 物理层 line termination（比特级接收）

先把线上来的信号/比特流接住。  chapter_4_notes

Step B: 链路层处理（例如 Ethernet receive）

把以太网帧这类 link-layer frame 做完接收端该做的事（解帧、校验、拿到 payload）。

 chapter_4_notes

Step C: lookup + forwarding（查表决定往哪发）

这一步是网络层数据平面的“灵魂”：  chapter_4_notes

- 读取到达数据报的头部字段（传统上主要看 **destination IP address**）
- 在输入端口本地的 **forwarding table** 里查：应该去哪个输出端口
- 这一步课件用一句话概括成 “**match plus action**”：匹配某些头字段 → 执行动作（转发到某输出端口/队列）。  chapter_4_notes

课件顺便点出两种风格：  chapter_4_notes

- **destination-based forwarding**：只看目的 IP（传统）
- **generalized forwarding**：可以看一组头字段（为后面 SDN/OpenFlow 埋伏笔）

Step D: queueing（可能排队）

如果后面“交换结构/输出端口”一时处理不过来，就会在输入端口产生排队。

 chapter_4_notes

这一小节你要抓住的 2 个“为什么”

1. 为什么转发表要 push 到 input port?

因为查表是每个包都要做的“超高频操作”，放在输入端口本地硬件/内存里，才能做到线速转发；routing processor 负责慢速的“算表/更新表”。  chapter_4_notes

2. 为什么 input port 里就会 queueing?

因为即使你查表很快，后面的 switching fabric/输出口也可能成为瓶颈，于是包只能先在输入端口等一等。  chapter_4_notes

快速自检（你答完我就继续讲 switching fabric 三种实现 + HOL blocking）

1. 你用一句话区分：routing processor 在做什么 vs input port lookup 在做什么？

 chapter_4_notes

2. 课件说“decentralized switching: lookup using forwarding table in input port memory”，你怎么理解 “decentralized” 在这里指什么？ chapter_4_notes

来源

     ...

1. 算表/更新表 vs. 根据ip等信息查转发目的地
2. 不清楚

你第 1 条说得完全对。  chapter_4_notes

第 2 条“不清楚”很正常，这个词在课件里其实是个**架构设计点**，我用最贴近图的方式解释一下。

“decentralized switching” 里的 decentralized 到底指什么？

课件那句是：**decentralized switching: lookup using forwarding table in input port memory**。  chapter_4_notes

这里的 **decentralized**（去中心化）指的是：

以前（更“中心化”的直觉）

- 包来了以后，可能要交给一个“中心”去做关键决定（比如由路由处理器/中心模块查表、决定输出端口），再把决定发回去。
- 问题：每个包都去找“中心”问一次，会变成瓶颈（路由器端口速率越高越扛不住）。

现在（课件强调的 decentralized）

- 每个 input port 自己就带着 forwarding table 的副本（或关键部分），包一到输入端口就能**本地查表**决定要送哪个输出端口。  chapter_4_notes
- routing processor 的角色变成：**慢速地算/更新表**，然后把表“分发/下发”到各个输入端口，而不是对每个包做实时查询。  chapter_4_notes

一句话：“**查表决策**”不再集中在一个中心点，而是分散到每个输入端口并行做。

 chapter_4_notes

这也解释了为什么课件把 lookup+forwarding 画在 input port 里：为了线速转发（line-rate），每个端口都要能独立干活。  chapter_4_notes

你可以用一个类比确认理解

- **中心化**：全班同学做题都要排队问老师“这题怎么做”。
 - **去中心化**：老师把“答案表/规则”发给每个同学，同学自己查；老师只负责更新规则。
-

小自检（只要一句话）

如果我说：“routing processor 不参与每个包的实时转发决策，只负责把转发表算好并下发。”——这句话对应的就是 centralized 还是 decentralized？  chapter_4_notes

你答完我就继续下一节：**switching fabric** 的三种实现 (**memory/bus/crossbar**)，以及很关键的 **HOL blocking** (输入排队导致的头阻塞)。

来源



decentralized

对 ☒ 那我们继续 4.2 里 router 内部的下一块：**Switching fabric** (交换结构)。

chapter_4_notes

Switching fabric 是干嘛的？

它就是路由器内部的“搬运系统”：

把某个输入端口收到的 **packet**，从输入侧搬到正确的输出端口。 chapter_4_notes

课件通常按三种实现方式讲：**via memory / via a bus / via an interconnection network (crossbar)**。 chapter_4_notes

1) Switching via memory (通过内存交换)

想成“老式单核”：

- 输入端口把包拷贝进 **系统内存**
- CPU/内存控制器再把包从内存拷贝到对应的输出端口缓冲区
- 关键瓶颈：一次转发要过两次内存带宽 (写入 + 读出)，所以总吞吐受内存带宽限制。

chapter_4_notes

直觉：端口速率上来后，这种方式很快就扛不住。

2) Switching via a bus (通过共享总线)

想成“一条共享走廊”：

- 所有输入端口把包放到同一条 **共享总线** 上广播

来源



pause, 你还漏了datagram forwarding table那部分

已思考 25s >

对，你说得很关键：在讲 switching fabric 之前，课件先把 **Datagram forwarding table + Longest prefix matching** 讲清楚了（就是“输入端口查表”的核心细节）。我们把这节补上再往下走。  chapter_4_notes

Datagram forwarding table：路由器到底“查的表”长什么样？

1) 表的本质：把“目的地址范围”映射到“输出接口”

课件先用“地址范围 (from...through...)”的形式展示：  chapter_4_notes

- 如果目的地址落在区间 A → 走 **interface 0**
- 落在区间 B → 走 **interface 1**
- 落在区间 C → 走 **interface 2**
- 否则 → 走 **interface 3 (default)**

你可以把它理解成：路由器不会为每一个具体 IP 写一条规则，而是按“网段/范围”写规则。

 chapter_4_notes

2) 用前缀/通配符压缩表示（更接近真实路由表的写法）

下一页把上面的“连续范围”改写成 前缀 + 星号通配符：  chapter_4_notes

- 11001000 00010111 00010*** ***** → interface 0
- 11001000 00010111 00011000 ***** → interface 1
- 11001000 00010111 00011*** ***** → interface 2
- otherwise → interface 3

这里的 * 就表示“这一位可以是 0 或 1”，所以一个 pattern 其实代表一大坨地址。

 chapter_4_notes

Longest prefix matching: 为什么要“最长前缀匹配”?

关键问题：同一个目的地址，可能匹配到不止一条规则（因为有的规则更粗，有的更细）。

课件给的原则是：  chapter_4_notes

对于给定目的地址，选择 **匹配前缀最长** 的那条（也就是“最具体”的那条）。

用课件的例子直观走一遍

课件给了这个目的地址（DA）：

11001000 00010111 00011000 10101010  chapter_4_notes

它会匹配两条：

- 匹配 ... 00011000 ***** （第三个字节完全固定）
- 也匹配 ... 00011*** ***** （第三个字节只固定前 5 位是 00011）

但 00011000 ***** **更“长”（更具体）**，所以最终选 **interface 1**。  chapter_4_notes

另一个例子（课件问 “which interface?”）：

11001000 00010111 00010110 10100001

它落在 ... 00010*** ***** 那条，所以走 **interface 0**。  chapter_4_notes

你现在应该形成的“查表动作”心智模型

输入端口看到一个目的 IP：

1. 找出所有能匹配的前缀项
2. 如果有多个，选 **最长前缀**
3. 得到一个输出接口号（0/1/2/3），把包交给 switching fabric 往对应 output port 搬

 chapter_4_notes

小自检（答完我就无缝接回 switching fabric + HOL blocking）

对目的地址 11001000 00010111 00011110 00000001，它会匹配哪条前缀？最终走哪个 interface？（不用算很多，只要看第三个字节是 00011110 属于哪一类）  chapter_4_notes

来源

对，走 **interface 2**   chapter_4_notes

因为第三个字节 00011110 的前 5 位是 00011，匹配 ... 00011*** ***** 那条，而它又比 default 更长，所以选它。  chapter_4_notes

那我们接回你刚才暂停的位置：**switching fabric**（三种实现）+ 为什么会出现输入排队和 HOL blocking。  chapter_4_notes

Switching fabric 三种实现（课件的经典三连）

1) Switching via memory（通过内存交换）

像“把包搬进中心仓库再搬出来”：

- 输入端口写入内存，输出端口再从内存读出
- **瓶颈：内存带宽**（一次包通常要写+读两次）  chapter_4_notes

2) Switching via a bus（通过共享总线）

像“所有人共用一条走廊”：

- 输入端口把包放上共享 bus，目标输出端口把它取走
- **瓶颈：总线一次只能让一个包通过**（大家争用）  chapter_4_notes

3) Switching via an interconnection network（通过互联网络/交叉开关 crossbar）

像“多条并行通道的矩阵”：

- 允许多个输入→多个输出**并行搬运**（只要它们不抢同一个输出）
- 因此可扩展性最好，是高性能路由器常见思路  chapter_4_notes

到这里你不用背硬件细节，只要记住**瓶颈从 memory → bus → crossbar 逐步缓解**。

输入排队（input queueing）为什么出现？

即使 lookup 很快，仍会卡在“能不能被交换结构/输出端口及时接走”上。

尤其当 **多个输入端口同时想把包发往同一个输出端口** 时，必然有人要等。

 chapter_4_notes

关键概念：HOL blocking（Head-of-Line blocking，队头阻塞）

这是这节最容易考、也最容易迷糊的点：

在输入端口采用 FIFO 队列时，队头那个包如果暂时过不去，会把它后面本来“可以过去”的包也堵住。  chapter_4_notes

用最直观的场景理解

- 输入端口队列头部的包要去 **output 1**，但 output 1 正在忙（或正被别的输入占用）
- 队列里第二个包其实要去 **output 2**，按理 output 2 此刻是空的、可以走
- 但因为 FIFO：必须等队头先走，所以第二个包也被迫等
- 这就是“头阻塞”：阻塞发生在队头，但伤害扩散到整条队列吞吐  chapter_4_notes

你可以把它记成一句话：

“队头堵车，后面所有车都动不了，即使旁边那条路其实是空的。”

小自检（两句话就行）

1. HOL blocking 发生的前提是什么队列策略？（FIFO 还是优先级/按输出分队列？）
2. 为什么 crossbar 能缓解（但不能彻底消灭）拥塞？你用“并行搬运”解释一下就行。

 chapter_4_notes

来源



1. fifo 2. 并行搬运是并行多个 queue 的包，同一个 queue 内仍然可能 block？

1) 对，HOL blocking 的经典前提就是输入端口是 FIFO 单队列。  chapter_4_notes

2) 你这句话已经非常接近“标准答案”了，我帮你把逻辑钉死：

- **crossbar 的优势**：在同一个时隙里，可以让“不同输入 → 不同输出”的多条传输并行发生，所以总体吞吐更高。  chapter_4_notes
- 但它不能彻底消灭阻塞/拥塞，原因有两个（都跟“争同一个输出”有关）：

1. 如果很多输入端口同时都想去 **同一个 output**，那这一时隙也只能选其中一个（其他仍得排队）。
2. 如果输入端口内部还是 **FIFO 单队列**，队头包卡在“热门输出”上，就会继续触发 **HOL blocking**：后面的包即使要去别的空闲输出，也被队头挡住了。

📄 chapter_4_notes

所以你可以把 crossbar 的作用总结成一句话：

它解决的是“交换结构本身能不能并行搬运”，但不自动解决“输入端口 FIFO 导致的队头阻塞”和“多个输入争同一输出”的竞争。 📄 chapter_4_notes

如果你这里没问题，下一小节我们就顺着课件讲 **output ports + output queueing + packet scheduling**（如 FIFO / priority / RR 等），以及延伸到 丢包发生在哪里、为什么。继续吗？

来源



继续

已思考 57s >

好，我们继续 4.2 What's inside a router 里下一节：**Output ports（输出端口）+ output queueing（输出排队）+ buffering（缓冲大小）**。

Output ports：为什么也要缓冲？会丢包吗？谁决定先发哪个包？

课件在 “Output ports” 这一页给了 3 个结论： 📄 chapter_4_notes

1) 为什么输出端口需要 buffer？

因为 从 **switching fabric** 到达输出端口的速率，可能在某些时刻 快于输出链路的发送速率（output line transmission rate）。

于是包会暂时堆在输出端口的 **datagram buffer / queue** 里等待发送。 📄 chapter_4_notes

直觉：交换结构像“内部高速搬运”，但真正发到外面的那条链路（网线/光纤）速度是固定的；内部一下子送来太多，就只能排队。

2) 输出端口也会丢包 (loss)

课件明确写: **Datagrams can be lost due to congestion, lack of buffers.**

📄 chapter_4_notes

也就是: 如果输出端口的缓冲区满了, 新到的包就会因为 **buffer overflow** 被丢掉。

📄 chapter_4_notes

3) 谁决定“队列里先发哪个包”?

由 **scheduling discipline (调度/排队策略)** 决定: 从输出队列里挑一个包发送。

📄 chapter_4_notes

课件还点了一个现实议题: **Priority scheduling vs network neutrality** (优先级调度 vs 网络中立)。 📄 chapter_4_notes

你现在先把它理解成: 如果允许“有的流量插队”, 那就涉及公平性/中立性问题。

Output port queueing: 它和 input queueing 的差别是什么?

课件对输出排队的定义很直接: 📄 chapter_4_notes

- 当 **arrival rate via switch (从交换结构过来的到达速率) > output line speed (输出线速)**
 - 就会 **buffering / queueing delay**
 - buffer 满了就 **loss (丢包)**。

对比你刚学的 input queueing:

- input 更像是“进门处塞车” (fabric 跟不上、或争同一输出导致 HOL blocking)
📄 chapter_4_notes
- output 更像是“出门口塞车” (出去那条线速不够) 📄 chapter_4_notes

How much buffering: 缓冲区到底该配多大?

课件给了两个经验法则: 📄 chapter_4_notes

Rule of thumb (RFC 3439)

平均缓冲大小 (buffering) $\approx \text{RTT} \times C$

- RTT: 典型往返时延 (课件举例 250ms)
- C: 链路容量 (例如 10Gbps)
 - 例子: $0.25\text{s} \times 10\text{Gbps} = 2.5 \text{ Gbit buffer}$ 📄 chapter_4_notes

Recent recommendation (有 N 条 TCP flows 时)

缓冲大小 $\approx RTT \cdot C / \sqrt{N}$  chapter_4_notes

你现在只要抓住直觉：

- 流越多（N 越大），每条流“分摊”拥塞波动，单链路需要的 buffer 不一定要像 $RTT \cdot C$ 那么夸张。

小自检（答完我们就进入下一节：4.3 IP datagram format / fragmentation）

1. 用一句话说：output queueing 发生的直接条件是什么（速率关系）？

 chapter_4_notes

2. 丢包（loss）在这部分课件里最直接的原因是什么？发生在哪个地方？

 chapter_4_notes

来源



1. 从交换fabric到达buffer的速率大于output port的
transmission rate
2. buffer满了，发生在output buffer里；
问题：input queueing都发生在input port或者fabric里是嘛

你 1/2 都答对   chapter_4_notes

你问：input queueing 都发生在 input port 或者 fabric 里吗？

input queueing 发生在哪里？（按课件图的说法）

“排队”这个动作发生在 input port 的输入缓冲区/输入队列里。  chapter_4_notes

原因通常是：fabric（交换结构）没法立刻把队头包送到目标 output（比如多个输入抢同一个输出、或者 fabric 本身吞吐受限），于是包只能先在 input port 等。  chapter_4_notes

所以更精确的说法是：

- queue 在 input port（排队的位置）
- bottleneck/争用可能在 switching fabric 或在目标 output 的“可用性”上（导致排队的原因）  chapter_4_notes

而你之前学的 **HOL blocking** 也是发生在 **input port** 的 **FIFO 队列**：队头去不了 → 后面都被堵住。  chapter_4_notes

如果你这里清楚了，我们下一节就按课件进入 **4.3 IP: the Internet Protocol**（IP 数据报格式、IPv4 地址、子网、CIDR、以及 fragmentation）。你想先从 **IP datagram format**（头部字段）开始吗？

来源



所以fabric到outputport的通道也是bottleneck对吗

对，**可能是 bottleneck**，但要看的哪种交换结构、以及当时的竞争情况。用你课件的“速率关系”语言总结一下：

- 如果 **switching fabric** 的总吞吐/仲裁能力 不够（例如 memory/bus 架构，或 crossbar 但很多输入同时抢同一输出），那么 **input → fabric → output 这段内部通道**就会成为瓶颈：包进不了/过不去，于是堆在 **input queue**。  chapter_4_notes
- 即使 fabric 很强（比如 crossbar 可并行），也仍可能因为 **同一时刻多个输入想去同一个 output**，导致“到该 output 的内部通道/连接”只能让一部分先过，剩下的继续在输入端口排队（这也是为什么 crossbar 只能缓解、不能消灭排队）。  chapter_4_notes

和输出端口 bottleneck 的区别是：

- **内部 bottleneck（fabric/仲裁）** → 主要体现为 **input queueing / HOL blocking**
- **外部 bottleneck（output link 线速）** → 主要体现为 **output queueing / output buffer overflow**

如果你 OK，我们就继续到 **IP datagram format**（IPv4 头部字段、为什么要有 TTL、checksum、fragmentation 字段）。

来源



继续

已思考 13s >

好，我们进入 **4.3 IP: Internet Protocol** 的第一小节：**IP datagram format (IPv4 头部字段)**。先把“这一张图”吃透，下一步再讲 fragmentation。

IPv4 数据报长什么样（头部 + 数据）

课件这页给的结构是：**IP header（固定字段 + 可选 options） + data（通常是 TCP/UDP segment）**。

你只要抓住一个核心：

路由器转发时主要看 **header**；payload（data）通常不看（只负责搬运）。

头部字段逐个解释（按课件顺序）

1) Version (ver)

IP 协议版本号，IPv4 就是 4。  chapter_4_notes

2) Header length (head. len)

头部长度的（字节）。因为有 options，所以头部不一定永远 20 bytes。  chapter_4_notes

3) Type of service (type of service)

可以理解为“这个包的服务类型/优先级/QoS 相关提示”。（这门课通常不深挖细分位的含义，你知道它是“分类/服务提示”即可。）  chapter_4_notes

4) Total length (length)

整个 datagram 的总长度（字节） = header + data。  chapter_4_notes

5) Identification (16-bit identifier)

分片用的编号：一个大包如果被切成多个 fragment，这些碎片会带同一个 ID，方便目的主机把它们认作“一组”。  chapter_4_notes

6) Flags + Fragment offset (flgs, fragment offset)

这俩是 fragmentation/reassembly 的核心字段：

- flags 里有“后面还有没有碎片”（more fragments 之类的含义）

- fragment offset 表示“这一片数据在原始大包里的位置”（注意 offset 的单位在后面分片例题里会出现）

（我们下一节讲 fragmentation 会把它算到你完全会。）

7) Time to live (time to live)

剩余可走的跳数 (hops)：每过一个路由器就减 1，减到 0 就丢弃。它的目的就是避免包在路由环路里无限转。

8) Protocol (upper layer)

告诉接收端：payload 该交给谁（TCP / UDP / ICMP 等）。

9) Header checksum (header checksum)

只校验 IP 头部（不是校验 payload）。因为 TTL 每跳都变，checksum 也得每跳重算/更新，所以设计上只保守地覆盖头部。

10) Source IP / Destination IP

32-bit 源地址、目的地址。路由器转发最关键的通常就是看 **目的地址** 去做 forwarding（最长前缀匹配）。

11) Options (if any)

可选：例如 timestamp、record route、指定必经路由器列表等（现实里不常用，主要因为会拖慢处理）。

这页最后一句很重要：overhead（开销）

课件提醒你：典型情况下

- TCP 头 20 bytes
 - IP 头 20 bytes
- **至少 40 bytes**，再加上应用层自己的开销。

这就是为什么“小包”比例高时，头部开销会很显著。

小自检（答完我就进 fragmentation 那两页例题）

1. TTL 为什么必须“每跳减 1”？如果不这么做会发生什么？  chapter_4_notes
2. “upper layer protocol” 这个字段的作用，用你自己的话说一句（它在接收端做什么分发动作）？  chapter_4_notes

[来源](#)

1. 可能让datagram在路由里loop转发
2. 做到底是给udp还是tcp的socket

两题都对✅（第2题你说得更细：它决定交给TCP/UDP/ICMP等，再由传输层按端口号分到具体socket）。

那我们继续：IP fragmentation（分片），这是IPv4特有的高频考点之一。

为什么需要 fragmentation?

因为不同链路有不同的 MTU（maximum transmission unit，最大传输单元）。

如果一个 IP datagram 太大，某一跳的链路装不下，就必须把它切成多个 fragments 才能过。

课件的关键规则是：

- 分片可以在中间路由器发生（IPv4 允许）
- 重组（reassembly）只在目的主机发生（路由器不帮你拼回去）

分片时头部里哪几个字段在起作用？

就是你刚见过的三个：

- **Identification**：同一个原始 datagram 的所有碎片都带同一个 ID
- **Flags**：通常关注 MF（more fragments）：是不是“后面还有碎片”
- **Fragment offset**：这一片数据在原始数据里的起始位置（按固定单位计）

以及每个 fragment 自己也有：

- **Total length**（它自己这片的总长度）
- **Header checksum**（每片都要对自己的头校验）

课件例题怎么做（最重要的计算套路）

课件给了一个典型场景：

原始 IP datagram 长度 **4000 bytes**，其中 **header 20 bytes**，数据 **3980 bytes**。
经过一条 **MTU=1500 bytes** 的链路，需要分片。

Step 1: 每个 fragment 最多装多少“数据”？

每个 fragment 都要带自己的 IP header (20 bytes)，所以每片最多数据是：

- $1500 - 20 = 1480 \text{ bytes data}$

Step 2: 把 3980 data 切成若干片

- $1480 + 1480 + 1020 = 3980$

所以会有 **3 个 fragments**：前两片满载 1480 data，最后一片 1020 data。

Step 3: 每片的 total length

- Fragment 1: $20 + 1480 = 1500$
- Fragment 2: $20 + 1480 = 1500$
- Fragment 3: $20 + 1020 = 1040$

Step 4: MF (more fragments) 怎么填

- 前两片后面还有碎片 → **MF=1**
- 最后一片后面没有 → **MF=0**

Step 5: Fragment offset 怎么算 (最容易错)

offset 的单位是 **8 bytes** (也就是“以 8 字节为粒度计数”)。

- Fragment 1 的数据从原始数据第 0 字节开始 → $\text{offset} = 0/8 = 0$
- Fragment 2 的数据从第 1480 字节开始 → $\text{offset} = 1480/8 = 185$
- Fragment 3 的数据从第 2960 字节开始 → $\text{offset} = 2960/8 = 370$

这里你会发现一个隐含条件：除最后一片外，数据长度最好是 8 的倍数 (1480 正好是 8 的倍数)，这样 offset 才是整数，设计就是为此服务的。

小自检 (你答完我们再往后走: IPv4 addressing / subnetting / CIDR)

如果把同一个原始 datagram (header 20, data 3980) 换成链路 MTU=1000:

1. 每片最多 data 是多少？
2. 至少要几片？ (不用算 offset 先)



pause, options字段不算在header的20byte立马

好问题，而且这是分片题里很容易踩坑的点。

结论：20 bytes 只是 IPv4 头部的“最小长度”。如果有 options，header 会更长，不一定是 20。

怎么判断/怎么用？

- IPv4 头部里有 **Header Length (IHL)** 字段，它告诉你“这个包的头部到底多长”。

 chapter_4_notes

- **options 如果存在，会被计入 header length。**所以：
 - 没有 options：header = 20 bytes（最常见）
 - 有 options：header > 20 bytes

所以你说得对：**不能无条件把 header 当 20**；只有在题目明确“header=20”或默认“无 options”时才这么做。课件的分片例题明确给了 header 20 bytes，所以才可以直接用 $1500 - 20 = 1480$ 。

还有一个细节：当一个 datagram 被分片时，**每个 fragment 都要带自己的 IP header**（同样包含 options 的话也会包含/反映在 IHL 里），因此算“每片能装多少 data”时，应当用 **MTU – 实际 header length**。

你这个点清楚了的话，我们可以回到刚才那道 MTU=1000 的自检题（或者你想先把 options 在分片时怎么处理再问也行）。

来源



1. 980； 2. 5

对 

- 每片最大 data： $1000 - 20 = 980$ bytes
- $3980 = 980 \times 4 + 60$ ，所以至少 5 片

顺便把这题完整补齐（以后考试常让你写表）：

每片 total length

- F1–F4: $20 + 980 = 1000$
- F5: $20 + 60 = 80$

MF 位

- F1–F4: MF=1
- F5: MF=0

Fragment offset（单位 8 bytes）

offset = “该片数据起始字节” / 8：

- F1: $0/8 = 0$
- F2: $980/8 = 122.5$ ❌（不是整数）

这一步会暴露一个关键约束：除最后一片外，每片 data 长度必须是 8 的倍数，否则 offset 算不成整数。课件前面的例子之所以用 1480，就是因为 1480 是 8 的倍数。

所以严格来说，在 MTU=1000 的情况下，实际会这样切：

- 每片 data 取不超过 980 的最大 8 的倍数：976
- $3980 = 976 \times 4 + 76 \rightarrow$ 仍是 5 片，但每片数据会是 976, 976, 976, 976, 76
这样 offset 才会是 0, 122, 244, 366, 488。

如果你对分片这里没问题，我们就继续下一节：IPv4 addressing / subnetting / CIDR（以及最长前缀匹配的地址表示法）。



pause, 每个 datagram, flag+offset 一共是 16 bits, MF 是 flags 其中一位, offset 一共几位

对，IPv4 头里 Flags + Fragment offset 这一块一共 16 bits。拆法是：

- Flags: 3 bits
 - 其中 1 bit 是 MF (More Fragments)
 - 还有 DF (Don't Fragment) 也在这里（另一位），以及 1 位保留位
- Fragment offset: 13 bits（所以 offset 一共 13 位）



继续

已思考 17s >

好，我们继续到 4.3 里的 IPv4 addressing（IPv4 地址 + 子网掩码怎么用）这一节，先把“地址怎么切 network/host、怎么用 mask 算 network/broadcast”讲透。

IPv4 地址：32 bits，两种常见表示

- IPv4 地址是 32 位。  chapter_4_notes
- 常见表示：
 - 点分十进制：192.168.1.1
 - 二进制：32 位从 MSB 到 LSB  chapter_4_notes

层次化结构：network part + host part

课件强调：一个 IP 地址会被切成两段：

- **network 部分**：标识“这个地址属于哪个网络/子网”
- **host 部分**：标识“该网络里的哪台主机”  chapter_4_notes

类比电话区号 + 号码：区号决定你在哪个区域，号码决定你是谁。  chapter_4_notes

Subnet mask：怎么从 IP 算出 network address?

课件给定义：Subnet mask 是一个 bit mask，用来从 IP 地址里算出网络地址，写法可以是：

- 255.255.0.0 这种点分
- 或 /16 这种前缀长度  chapter_4_notes

核心操作：Network address = IP AND subnet mask

课件示例：IP 137.189.11.73，mask 255.255.0.0 (/16)  chapter_4_notes

- mask 的前 16 位是 1，后 16 位是 0
- IP 与 mask 做 bitwise AND
 - host 位被清零
 - 得到 network address：137.189.0.0  chapter_4_notes

同一页还给了 broadcast address 的定义

- **network address**: host bits 全 0
- **broadcast address**: host bits 全 1

并且提醒: host 地址不能用“全 0 (this network)”或“全 1 (broadcast)”。

 chapter_4_notes

这跟路由/转发有什么关系?

课件说得很直接:  chapter_4_notes

- 同一 **subnet** 内的主机: 可以直接在链路层互通 (不需要路由器转发)
- 跨 **subnet**: 才需要路由器; 路由器的转发表把“某段 IP 范围/前缀”映射到某个输出接口

小自检 (答完我继续讲 CIDR: 为什么 classful 过时、/17 这种怎么读)

给你一个: IP 137.189.178.73/17 (课件后面会用这个例子)  chapter_4_notes

1. /17 意味着 network part 是多少位? host part 是多少位?
2. 你觉得它的 subnet mask 点分写法大概长什么样 (只要说“第三个字节从哪一位开始变化”也行)

来源

     ...

1. 17位, 15位 2. ?

已思考 11s >

继续: 这一小节就是 CIDR (Classless Inter-Domain Routing) / “/x” 前缀表示法, 以及你刚才卡住的 /17 的 subnet mask 怎么写。

从 classful 到 CIDR: 为啥要 “/17” 这种写法?

课件先说 classful (A/B/C) 的问题：网络位数是“隐含的”，不够灵活，导致地址浪费。

于是 CIDR 引入：网络位数不再由 A/B/C 固定决定，而是显式写在 subnet mask 里，比如 255.255.128.0 或 /17。  chapter_4_notes

并且强调一点：路由器之间需要交换 subnet mask (前缀长度)，不像 classful 里“看到 137.* 就默认 Class B”。  chapter_4_notes

“/17” 到底怎么变成点分 mask?

规则就一句话：

/17 表示 subnet mask 的前 17 位是 1，后面 15 位是 0。  chapter_4_notes

把 32 位按字节分组：

- 第 1 个字节：8 个 1 → 255
- 第 2 个字节：8 个 1 → 255
- 第 3 个字节：还需要 1 个 1 (因为 17=16+1)，剩下 7 个 0 → $10000000 = 128$
- 第 4 个字节：全 0 → 0

所以：

- /17 的 mask = 255.255.128.0
-

课件例子：137.189.178.73/17 (你要会完整走一遍)

课件直接把它算出来了：

- IP: 137.189.178.73
- mask: 255.255.128.0 (二进制第三个字节是 10000000)

Network address = IP AND mask

- 结果：137.189.128.0

Broadcast address (host bits 全 1)

- 结果：137.189.255.255

直觉上你也能看出来：

/17 只把第三个字节的最高位固定住 (0/1 分半块)，所以这个网络覆盖的第三个字节范围是 128–255 (对应最高位为 1)。

小自检（答完我继续：route aggregation + longest prefix match 那几页）

1. /17 下，137.189.50.1 会落在同一个 subnet 吗？为什么？（看第三个字

来源



pause，我对你上面的问题还有疑问呢

当然，可以——你具体卡在哪个点？我先把你可能的两种疑问都直接讲清楚，你对号入座就行（然后你再补充你真正的问题）。

你刚才那题（137.189.178.73/17）里最常见的疑问点

A. 为什么 /17 会把第三个字节“分成两半”（0–127 和 128–255）？

因为 /17 = 前 16 位（前两个字节）全固定 + 第 17 位（第三个字节的最高位）固定。

- 第三字节最高位为 0 ⇒ 范围 0–127
- 第三字节最高位为 1 ⇒ 范围 128–255

所以 178 (>127) 一定在“128–255 这半边”。

B. 为什么 network address 是 137.189.128.0，而不是 137.189.178.0？

network address 的定义就是 **host bits 全清零**。

在 /17 里，host bits 是：第三个字节剩下 7 位 + 第四个字节 8 位。

所以 178 的低 7 位会被清掉，只留下最高位 1，对应就是第三个字节变成 10000000 = 128；第四个字节清零变 0。

你现在的疑问更像 A 还是 B？还是别的（比如 broadcast 为什么是 255.255）？你把你困惑的那句话直接贴出来也行。



小自检（答完我继续讲 CIDR：为什么 classful 过时、/17 这种怎么读）

给你一个：IP 137.189.178.73/17（课件后面会用这个例子）

1. /17 意味着 network part 是多少位？ host part 是多少位？
2. 你觉得它的 subnet mask 点分写法大概长什么样（只要说“第三个字节从哪一位开始变化”也行）
是这个问题

明白了——你是说你**对自检第 2 问“mask 点分写法大概长什么样”**这一步没感觉。

我把这题用一个“你以后看到 /x 立刻能写出来”的方法讲清楚。

自检第 2 问：/17 的 subnet mask 点分写法怎么想？

一步法（最稳）

把 /17 拆成 “完整字节 + 剩余位数”：

- $17 = 8 + 8 + 1$
- 前 2 个字节全是 1 $\rightarrow$ 255.255
- 第 3 个字节只有 1 个 1（最高位），剩下 7 个 0 $\rightarrow$ 二进制 10000000 $\rightarrow$ 十进制 128
- 第 4 个字节全 0 $\rightarrow$ 0

所以 subnet mask = 255.255.128.0

你问的那句提示：“第三个字节从哪一位开始变化”是什么意思？

意思是：在 /17 下，

- 前 16 位已经固定（前两个字节都固定）
- 第 17 位也固定（第三个字节的最高位固定）
- 从第三个字节的第 2 位（也就是“次高位”）开始，直到第四个字节，全都是 host bits，会变化

所以你可以用一句话回答第 2 问：

/17 的 mask 是 255.255.128.0；第三个字节只有最高位固定（128 那一位），其余位都属于 host，会变化。

如果你愿意，我再给你一个 10 秒心算模板（选做）：

- /8 → 255.0.0.0
- /16 → 255.255.0.0
- /24 → 255.255.255.0
- 介于它们之间就看“多出来的那几位”：
 - /17 = /16 + 1 位 → 第三个字节变成 128
 - /18 → 192
 - /19 → 224
 - /20 → 240
 - /21 → 248
 - /22 → 252
 - /23 → 254

现在你能回答我刚才追问的那题了：

在 /17 下，137.189.50.1 和 137.189.178.73 同一个 subnet 吗？为啥（看第三个字节最高位是 0 还是 1）？



不是

已思考 20s >

对，不是同一个 subnet。在 /17 里，决定“属于哪半个网段”的关键就是第三个字节的最高位：

- 178 的二进制是 10110010，最高位是 1 → 落在 137.189.128.0/17 这半边（128–255）  chapter_4_notes
- 50 的二进制是 00110010，最高位是 0 → 落在 137.189.0.0/17 这半边（0–127）
所以不在同一个 subnet。  chapter_4_notes

下一节：Hierarchical addressing 的核心用途——route aggregation + longest prefix match

1) ISP 分配地址块：对外“汇总”，对内“细分”

课件的故事是：ISP（例如 Fly-By-Night-ISP）拿到一整块地址 `200.23.16.0/20`，然后把它切成多个更小的块（例如每个组织一个 `/23`）：`200.23.16.0/23`、`200.23.18.0/23`、`200.23.20.0/23` ...

关键点：外部世界不需要知道 ISP 内部切了多少块。外部路由器只要记一条“汇总路由”就行：

“发往 `200.23.16.0/20` 的都朝 Fly-By-Night-ISP 走”。  chapter_4_notes

这就是 **route aggregation（路由聚合）**：减少全网路由表规模。  chapter_4_notes

2) 但如果出现更“特殊”的走法：就加一条更具体的路由

课件接着问：如果 Fly-By-Night-ISP 收购了 ISPs-R-Us，并让 **Organization 1** (`200.23.18.0/23`) 通过子公司 ISPs-R-Us 接入互联网，会怎样？  chapter_4_notes

这时，外部某些路由器可能同时看到两条信息：

- 聚合路由： `200.23.16.0/20` → Fly-By-Night-ISP
- 更具体路由： `200.23.18.0/23` → ISPs-R-Us  chapter_4_notes

用 **longest prefix matching** 解决冲突：永远选更具体（前缀更长）的那条。

3) Longest prefix match 的“纯查表版”例子

课件给了 `192.168.1.140` 这个例子，同时匹配：

- `192.168.0.0/16`
- `192.168.1.0/24`
- `192.168.1.128/25`

最后选 `/25`（前缀最长，最具体）。  chapter_4_notes

小自检（你答完我再继续讲 DHCP：主机怎么拿到 IP/网关/DNS）

1. `192.168.1.100` 会选上面三条里的哪一条？（`/16`、`/24`、还是 `/25`？）

 chapter_4_notes

2. 去往 `200.23.18.5` 时，如果同时存在 `200.23.16.0/20` 和 `200.23.18.0/23` 两条路由，会走哪条？为什么？  chapter_4_notes

来源



你还漏了classful routing这部分

已思考 11s >

对，你指出得对：刚才我直接跳到 CIDR 了，但课件在 CIDR 前面确实先讲了 classful IP + classful routing（现已过时）。我们把这一节补齐。

4.3 IPv4 addressing: Classful（有类地址）与 Classful routing

1) Classful IP 的核心：靠“前几位”决定这是哪一类

课件把 IPv4 的“原始层次结构（现在 obsolete）”列成 A/B/C/D/E 五类：  chapter_4_notes

- Class A：大型组织
- Class B：中型组织
- Class C：小型组织
- Class D：组播
- Class E：研究用途

关键句：“网络位数（network bits）是 implicit（隐含的），由 prefix coding（前缀编码）决定”。  chapter_4_notes

也就是：你看到一个 IP 的开头比特，就“默认”它网络部分有多少位（因此默认 subnet mask 是多少）。

课件还在例子里点出来：137 = 10001001b，前两位是 10，所以它是 Class B。

 chapter_4_notes

2) Subnet mask 在 classful 里“默认就知道”

课件用 137.189.11.73 演示：它是 Class B，所以默认 mask 是 255.255.0.0 (/16)，网络地址就是 AND 后的 137.189.0.0。  chapter_4_notes

并且强调：network address 的 host bits 全 0，broadcast address 的 host bits 全 1；主机不能用全 0 或全 1 的 host 地址。  chapter_4_notes

3) 什么是 Classful routing?

课件在 “Relation to Routing/Forwarding” 里写得很直白：路由器转发表把 IP 地址范围映射到输出接口，而在 classful 里，“范围指示器”就是 network address part；并且 subnet mask 是 implicit 的。  chapter_4_notes

配图示例：

转发表里可以直接写：

- 137.189.0.0 → e0/0
- 137.190.0.0 → e0/1

因为大家都默认它们是 Class B (/16)，不需要在路由表里再写 /16 。

 chapter_4_notes

一句话总结：

classful routing = 不在路由更新/路由表里显式携带 mask，靠地址所属的 Class 来“默认推断”网络前缀长度。  chapter_4_notes

4) 为什么 classful 过时？（课件列的痛点）

课件列了三条核心问题：  chapter_4_notes

- Class C 往往太小（最多 254 hosts），Class B 又太大（最多 65534 hosts）
- Class A/B 利用率低，浪费大量地址空间
- 地址分配缺乏灵活性

这就是 CIDR 出现的动机：去掉固定的网络位数限制，让网络位数由显式 mask 或 /x 决定，并且“路由器之间需要交换 subnet masks”。

你现在可以提问/自检（我不继续往下讲 DHCP，先等你确认）

两个快问：

1. 在 **classful routing** 里，为什么路由器看到 137.189.0.0 这种网络号就能知道它对应 /16？（提示：看 137 的二进制前缀）
2. CIDR 相比 classful，课件明确说“路由器之间需要交换什么新信息”？  chapter_4_notes

来源



ChatGPT 5.2 Thinking

   ...完全正确 

那我们回到你刚才已经进入的主线：**route aggregation + longest prefix matching**，然后顺着课件通常就会到 **DHCP**（主机如何拿到 IP/网关/DNS）。

Route aggregation（路由聚合）：为什么能让全网路由表变小？

ISP 拿到一个大的地址块（比如 `/20`），对外只公告这一条“汇总前缀”；ISP 内部再把它切给不同组织（`/23`、`/24` ...）。外部路由器只需要记住“去这一大块都往 ISP 走”，不用知道内部细分。  chapter_4_notes

Longest prefix matching：汇总 + 例外如何共存？

当同时存在：

- 大的汇总前缀（更粗）
- 某个子网的更具体前缀（更细）

转发时永远选**前缀更长**的那条。这样就实现了：

- 默认走汇总路径
- 对某个子网可以“特例绕路”而不破坏汇总

回到刚才我给你的两个自检题（你现在应该能秒答）

1. `192.168.1.100` 在 `/16`、`/24`、`/25` 三条里选哪条？（提示：`/25` 覆盖 128–255 还是 0–127？）  chapter_4_notes
2. 同时存在 `200.23.16.0/20` 和 `200.23.18.0/23`，去 `200.23.18.5` 走哪条？为什么？  chapter_4_notes

[来源](#)     ...

1. /24; 2. /23

对 

1. 192.168.1.100 : 不在 192.168.1.128/25 (那是 128–255), 所以匹配到最具体的是 192.168.1.0/24 。  chapter_4_notes
2. 去 200.23.18.5 : 同时匹配 /20 和 /23, 选更具体的 200.23.18.0/23 (最长前缀匹配)。  chapter_4_notes

接下来按课件顺序通常就是 DHCP: 主机如何自动获得 IP 地址、子网掩码、默认网关、DNS。我们先讲 DHCP 的第一段: “为什么需要 DHCP + 四步流程”。

DHCP 是干什么的?

它解决的问题是: 一台主机刚连上网络时, 需要这些配置才能正常上网:

- 自己的 IP address
- subnet mask
- default gateway (第一跳路由器地址)
- local DNS server 地址 (如果有)

DHCP 做的就是 自动分配/租赁 (lease) 这些信息。

DHCP 的 4 步 (DORA)

课件用 4 条消息描述 (你可以把它当成“简化版握手”):

1. DHCP discover: 客户端广播“我来了, 有没有 DHCP server? ”
2. DHCP offer: 服务器回应“我可以给你这个 IP (以及配置)”
3. DHCP request: 客户端说“我接受这个 offer, 请把它租给我”
4. DHCP ack: 服务器确认“OK, 这个租约生效 (含 lease time)”

注意: 一开始客户端还没有 IP, 所以 discover 必须用广播方式发出去 (你之前在另一段复习里也碰到过 255.255.255.255)。

小自检 (答完我继续: DHCP 报文里到底装了什么、为什么要 request/ack 两步)

1. 为什么 DHCP 的第一步要用广播，而不是单播给服务器？
2. 你能用一句话解释“lease time（租期）”是什么意思吗？

来源



1. 因为自己还没有ip地址2. dhcp给该client分配的该ip配置的可用时间？

对 ☒ （第 1 题再补一句：不仅没 IP，也不知道 DHCP server 在哪，所以必须广播；第 2 题就是“租约有效期”）。

我们继续下一小节：DHCP 报文“到底携带什么配置”、以及 DORA 里 request/ack 为什么不能省。

DHCP 给你的不仅是 IP：还有一揽子“开机上网必需品”

课件列的典型内容是：

- assigned IP address（分配的地址）
- subnet mask（子网掩码）
- address of first-hop router / default gateway（默认网关）
- address(es) of local DNS server(s)（DNS 服务器）
- lease time（租期）

所以你可以把 DHCP 想成：主机的“网络配置自动下发系统”。

为什么有 request + ack 两步？（不是 offer 就完事）

直觉：offer 只是“我愿意给你”，request/ack 才是“这次分配真的落地”。

更具体一点，常见理由是：

1. 可能有多个 DHCP server 同时 offer

客户端需要用 request 明确告诉大家：我选了哪一个 offer，其他服务器就可以把自己预留的地址释放回池子。

2. 服务器需要最终确认并记录租约状态

ack 是服务器的“最终提交”：把这次租约写进自己的状态（并把完整配置确认发给客户端）。

你可以把 DHCP 的状态机记成一句话

discover 找人，offer 给方案，request 选方案，ack 落合同。

小自检（这两题做完你就能把 DHCP 和后面的 DNS/ARP 串起来）

1. DHCP 完成后，你的主机为什么还经常需要 ARP？（提示：要把“某个 IP”变成“某个 MAC”才能发以太网帧）
2. DHCP 拿到 DNS server 地址之后，浏览器访问网页前，下一步通常会先干什么？（一句话：把域名变成什么）

📄 🍷 🗨️ ⬆️ ↺️ ...

没在课件里看到arp

已思考 13s >

你看得很细：这份 **Chapter 4 slides** 里确实没讲 **ARP**（至少在我检索到的内容里没有出现 ARP/Address Resolution）。我们就严格按课件走，把我刚才那句“ARP”当成课外串联理解即可，不把它当本章必考点。

那我们继续课件在 **DHCP / addressing** 之后的下一小节：**IP addressing: the last word + IPv4 地址短缺 + 私有地址 RFC1918 + NAT 引入**。

4.3 IP addressing: the last word（地址块从哪来？）

课件的问答是：ISP 的地址块最终谁来分配？答案：**ICANN**（Internet Corporation for Assigned Names and Numbers），负责地址分配、DNS 管理、域名相关事务。

📄 chapter_4_notes

IPv4 Address Shortage：为什么会有 NAT？

课件直接说：**IPv4 地址空间已经分配完**；而现实里很多网络“主机数 > 可用公网 IP 数”。

📄 chapter_4_notes

所以就引出两个关键概念：

1) 私有地址（RFC1918）

课件列了三段经典私网地址范围（这些地址在公网不会被路由转发，ISP 会丢弃带私网源/目的的包）：

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

2) NAT（Network Address Translation）

课件用“家庭网络”画图：LAN 内部机器用 10.0.0.x 这种私网地址，但出网时，**整个 LAN 对外只呈现一个公网 IP（NAT IP），并用端口号区分不同内部连接。**

NAT 在做什么（用课件那张“表”来理解）

NAT 路由器维护一个 **NAT translation table**，把内部的（LAN 内 IP，LAN 内端口）映射到（公网 IP，新端口）：

- 出站包：改写 **源 IP/源端口** → 公网 IP + 新端口，并在表里记一条映射
- 入站包：根据目的端口查表，把 **目的 IP/目的端口** 改回去 → 送到对应内网主机/端口

课件还强调：因为端口是 16-bit，所以一个公网 IP 可以支撑很多并发连接（量级 ~6 万）。

📄 chapter_4_notes

小自检（两题，答完我就继续讲 NAT 的争议点/end-to-end，再往 IPv6 走）

1. 为什么说“私有 IP 的包在公网会被 ISP 丢弃”，这对私网主机直接上网意味着什么？

📄 chapter_4_notes

2. NAT 表里那条映射里，“新端口”起到的作用是什么？（一句话：用来区分谁）

来源



不清楚

没事，这一段确实容易“看懂图但不明白含义”。我把你刚才两题用最直观的方式讲清楚（仍然严格贴着课件这几页：私有地址 + NAT 表）。

1) “私有 IP 的包在公网会被丢弃”是什么意思？对上网有什么影响？

课件列的私有地址段（RFC1918）是：10/8、172.16/12、192.168/16。

📄 chapter_4_notes

这些地址的特点是：只允许在“一个组织/家庭内部网络”使用，互联网骨干/ISP 的路由器不会为它们提供全网可达的路由，通常还会直接过滤（丢弃）带这些源/目的的包。

📄 chapter_4_notes

所以影响就是：

- 如果你一台内网主机的源地址是 10.0.0.5，你把包直接发到公网：
公网路由器看到“源是私网地址”，不会帮你转发/也没法把回包正确送回 10.0.0.5
（因为全球有无数个不同家庭都在用 10.0.0.5）。📄 chapter_4_notes
- 结果：私网主机不能“直接用自己的私网 IP”在公网通信。想上网，就需要 NAT 把它“伪装成一个公网 IP”。

一句话记忆：私网地址在互联网里“不唯一、不可路由”，所以必须经过 NAT 才能上网。

2) NAT 表里的“新端口”到底干嘛？（为什么需要它）

课件的 NAT translation table 本质是在做“复用”：

多台内网机器共享 同一个公网 IP 出网时，用不同的 公网端口号 区分“回包该给谁”。

举个和课件表一致的直觉例子（只看思想）：

- 内网 A：10.0.0.1:3345 访问某网站
 - 内网 B：10.0.0.2:3345 也访问某网站
- 它俩在内网端口都可能一样（都叫 3345），没问题。

但到了公网侧，它们都要共享同一个公网 IP，比如 138.76.29.7，这时 NAT 就给它们分配不同的“新端口”：

- 10.0.0.1:3345 → 138.76.29.7:5001

- 10.0.0.2:3345 -> 138.76.29.7:5002

这样网站回包发到：

- 138.76.29.7:5001 -> NAT 查表 -> 转回 10.0.0.1:3345
- 138.76.29.7:5002 -> 转回 10.0.0.2:3345

一句话：新端口用来在“同一个公网 IP”下区分不同内网连接/主机（多路复用）。

快速自检（你答完我就继续讲 NAT 的争议点 + IPv6）

1. 只用一个公网 IP，NAT 还能让很多内网连接同时上网，靠的主要是“改写哪个字段”？（IP？端口？还是两者）
2. 如果外部回包到达 NAT，但 NAT 表里没有对应端口映射，会发生什么？（丢弃/无法转发给内网谁）

来源



1. IP和端口2. 丢弃

已思考 10s >

好，我们继续下一小节：NAT is controversial（NAT 的争议点）→ 引出 IPv6 动机。这一段课件信息密度不大，但很爱考“概念题”。

NAT 为什么“有争议”？（课件给了 3 条）

课件直接列了三点：

1) “routers should only process up to layer 3”

路由器按分层思想，本来只需要处理到网络层（L3）就行：看目的 IP、转发。

但 NAT 为了复用公网 IP，必须看/改传输层端口号（L4），还要维护一个映射表（状态）。

这被认为“破坏了简单的分层/透明转发”。  chapter_4_notes

2) “violates end-to-end argument”

端到端原则（你可以粗略理解为）：网络内部尽量简单，复杂功能尽量放在端系统。

NAT 让网络中间节点做了很多“智能改写 + 维护状态”的事，并且会影响连接建立方式，因此被认为违反端到端思想。  chapter_4_notes

3) 应用开发者必须考虑 NAT（尤其 P2P）

课件点名：**P2P 应用**要考虑 NAT 的存在。  chapter_4_notes

直觉就是：在 NAT 后面的主机不“**显式可达**”，外部想主动连进来通常会很麻烦（因为 NAT 表里可能没有映射就会丢弃你刚才答的那种入站包）。

最后课件总结了一句立场：

地址短缺更“应该”由 **IPv6** 来解决，而不是长期靠 NAT。

小自检（答完我就正式进入 IPv6：动机 + 40B 头 + 不允许分片）

1. 你用一句话说：NAT 为什么会被说成“路由器不该处理到 L4”？（关键词：端口、改写、状态表）  chapter_4_notes
2. 结合你前面说的“表里没映射就丢弃”，为什么 NAT 会让 **外部主动连入内网主机**变麻烦？

来源



1. 要看端口和改写端口 2. 涉及在isp路由器处改ip和端口

1) 对，补全成一句更“考试版”的话就是：**NAT 路由器不仅看 L3 地址，还要看/改 L4 端口并维护映射状态，因此不再是“纯 L3 转发设备”。**  chapter_4_notes

2) 你抓到“改 IP/端口”了，再把“为什么外部连入麻烦”的关键补上：**外部发来的入站包如果没有对应的 NAT 映射（port mapping），NAT 就不知道该转给哪个内网主机，于是会丢弃。**所以“外部主动发起连接到内网”默认是不可达的，除非有预先配置/穿透机制。

好，现在进入下一节：**IPv6**（课件通常顺着 NAT 争议就引出 IPv6）。我们先讲 IPv6 的“**动机 + 头部格式关键变化**”。

IPv6：为什么要它？

课件核心动机：**解决 IPv4 地址短缺**（128-bit 地址空间巨大），并且在协议设计上做了一些简化/改进。  chapter_4_notes

IPv6 datagram format（你要抓住 3 个变化）

课件把 IPv6 头部画出来，你不需要背每一位，但要记住这 3 个对比点：

1) 地址从 32-bit → 128-bit

Source/Destination 都是 **128 bits**。  chapter_4_notes

2) 头部固定 40 bytes（更规整）

IPv6 的基本头部长度的固定，options 不像 IPv4 那样塞在可变头里，而是用 **extension headers** 的思路（课件强调“简化路由器处理”）。

3) 不在中间路由器分片（fragmentation 方式改变）

IPv6 的分片机制与 IPv4 不同：**路由器不负责分片**，分片相关由端系统/扩展头处理（课件用来对比 IPv4 的 fragmentation）。  chapter_4_notes

（你如果后面要做题，我们再把“哪些字段在 IPv4 里消失/改名”详细对照。）

小自检（答完继续讲：IPv4→IPv6 的过渡方式）

1. IPv6 头部为什么说“对路由器更友好/更易处理”？（提示：固定长度、少变动字段）
2. IPv6 为什么能从根子上缓解“地址短缺”而不依赖 NAT？（一句话：地址位数）

来源



长度固定；位数多

对 

那我们继续最后一小节：**IPv4 → IPv6 的过渡（tunneling）**。

IPv4-IPv6 transition: 为什么需要“过渡机制”?

现实不可能“一夜之间全网换成 IPv6”。所以在很长一段时间里会出现：

- 有些网络/路由器只支持 IPv4
- 有些已经支持 IPv6
- 两者要互通，就需要过渡方案

课件重点：Tunneling（隧道）

课件给的核心思想是：

让 IPv6 数据报在一段 IPv4 网络中“穿越”，方法是把 **IPv6 datagram 当作 payload**，再套一层 **IPv4 header**（封装），在 IPv4 网络里当成普通 IPv4 包转发；到隧道出口再解封装，恢复成 IPv6 继续走。

三步你要会讲清楚

1. 入口路由器（tunnel entry）：把 IPv6 datagram **encapsulate** 到 IPv4 datagram 里
2. 中间 IPv4 网络：只看外层 IPv4 头，照常转发（它不需要懂 IPv6）
3. 出口路由器（tunnel exit）：把外层 IPv4 头去掉（decapsulate），拿回 IPv6 datagram 继续按 IPv6 转发

你可以把它类比为“在不兼容的路段把货物装进一个更通用的集装箱”，穿过去再拆箱。

小自检（最后两题，答完 Chapter 4 的主线基本就通了）

1. 在 tunneling 过程中，IPv4 网络里的路由器看到的“目的地址”是谁的地址？是 IPv6 的 destination 还是外层 IPv4 header 的 destination？
2. 为什么 tunneling 能让“不支持 IPv6 的中间网络”也帮你转发 IPv6 流量？用一句话解释。



1. ipv6网络的入口路由器地址，ipv4header的目的地
2. 包了ipv4的头

2) 对 ☒：因为封装了一层 IPv4 头，中间 IPv4 网络把它当普通 IPv4 包转发就行。

1) 这里差一点点：中间 IPv4 网络里看到的“目的地址”是外层 IPv4 header 的 **destination**，而这个 destination 通常设成隧道出口（tunnel exit）的 IPv4 地址，不是入口路由器。

（入口负责“封装并把包发进隧道”，出口负责“解封装”。）

到这里 Chapter 4 的主线（路由器转发/队列、IPv4 地址与 CIDR、DHCP、NAT、IPv6、tunneling）已经顺完了。你想下一步继续哪块？常见两种：

- 做几道 **fragmentation/CIDR/longest-prefix** 的计算题巩固
- 或者继续下一章（control plane: routing algorithms / RIP/OSPF/BGP）

